

4. PostgreSQL Index Types

1. Hash Index

Overview

A Hash Index is a specialized PostgreSQL index type optimized for **equality comparisons (=)**.

Unlike B-tree indexes, which store values in sorted order, Hash Indexes store hash codes generated from indexed column values. Each hash code maps to a **bucket** containing references (TIDs) to matching table rows.

Hash indexes are most effective when queries primarily perform **exact-match lookups**.

Creating a Hash Index

```
CREATE INDEX idx_users_email_hash
ON users USING hash (email);
```

Example table:

```
CREATE TABLE users (
    id      BIGINT,
    email   TEXT
);

CREATE INDEX idx_users_email_hash
ON users USING hash(email);
```

Supported Operations

Hash indexes are designed specifically for **equality comparisons**.

Supported:

```
WHERE email = 'john@example.com'
```

Not supported:

```
WHERE email > 'john@example.com'
WHERE email < 'john@example.com'
WHERE email BETWEEN 'a' AND 'z'
ORDER BY email
WHERE email LIKE 'john%'
```

For these operations, PostgreSQL typically uses a B-tree index instead.

Internal Structure

Metapage

Bucket 0

Bucket 1

Bucket 2

Page 100

Page 101

Page 102

hash=0x12A
TID (1,5)
TID (1,8)
TID (2,3)

hash=0x8BC
TID (3,2)
TID (3,7)
TID (4,1)

hash=0x4F1
TID (5,1)
TID (5,4)
TID (6,2)

Overflow 200

TID (7,3)
TID (8,5)
TID (9,1)

Hash Collisions

Different values may produce the same hash code:

```
hash("ABC") = 12345  
hash("XYZ") = 12345
```

This is called a **collision**. Because of collisions, PostgreSQL verifies the actual row value after locating candidate rows.

```
SELECT * FROM users WHERE email = 'ABC';
```

Query execution process:

```
SELECT *  
FROM users  
WHERE email = 'john@example.com';
```

Hash Function

```
hash('john@example.com')
```

Bucket #5

Bucket Page

TID List
(12,3) (15,7)

Heap Rows

Verify Actual Value

Return Row

Collisions are expected and handled automatically by PostgreSQL.

When to Use Hash Index

Good candidates:

Use Case	Example
Email lookup	WHERE email = ?
Session token	WHERE session_token = ?
API key	WHERE api_key = ?
UUID	WHERE user_uuid = ?

Characteristics:

- Equality lookups dominate the workload
- High-cardinality values
- No range queries
- No sorting requirements

CRUD Performance Characteristics

Operation	Description
SELECT	Optimized for equality (=) lookups. In ideal conditions, hash-based access provides near O(1) lookup performance by directly locating the target bucket.
INSERT	Requires computing the hash value and adding a new index entry to the appropriate bucket, introducing additional write overhead.
UPDATE	If the indexed column is modified, PostgreSQL must remove the old hash entry and create a new one, making updates more expensive.
DELETE	Requires removing the corresponding index entry from the bucket, resulting in additional work compared to a table without an index.

Note: The theoretical O(1) lookup performance may degrade when buckets become heavily populated, collisions increase, or overflow pages need to be traversed.

Hash Index vs B-tree

Feature	Hash	B-tree
---------	------	--------

= (equality)	Yes	Yes
< > (range)	No	Yes
BETWEEN	No	Yes
ORDER BY	No	Yes
LIKE 'abc%'	No	Yes

Use a Hash Index when queries primarily perform exact-match lookups using the equality (=) operator and range-based operations are not required. For workloads involving sorting, range queries, or mixed access patterns, a B-tree Index is generally the better choice. In most PostgreSQL applications, B-tree serves as the default index type due to its versatility, while Hash indexes are best suited for specialized equality-based workloads.

2. GIN Index (Generalized Inverted Index)

Overview

A **GIN (Generalized Inverted Index)** is a PostgreSQL index type designed for columns whose values contain **multiple components** — such as array elements, JSONB keys, or text lexemes.

While a B-tree index handles simple scalar values like `WHERE age = 25`, a GIN index answers questions like `WHERE tags @> '{postgresql}'` — “does this array contain this element?”

The core idea: instead of indexing a row as a single value, GIN **indexes each individual component** of that value separately.

Internal Structure

GIN consists of two main parts:

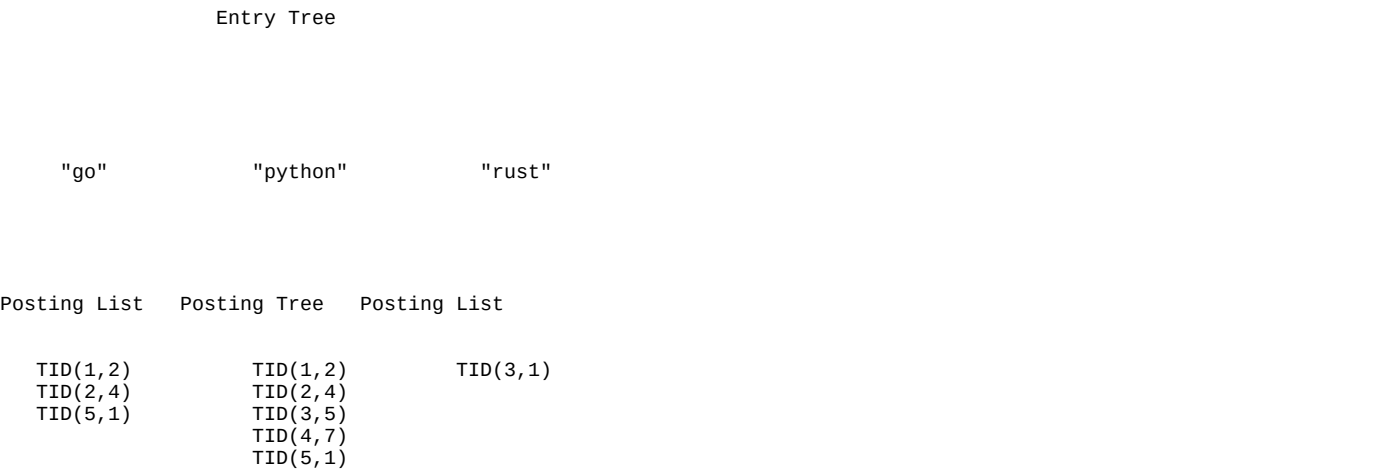
Entry Tree

All unique components across all indexed rows are stored in a B-tree structure. For example, if three rows contain arrays `{go}`, `{python}`, and `{rust}`, the entry tree holds `go`, `python`, and `rust` as separate keys.

Posting List / Posting Tree

Each key in the entry tree holds references (TIDs — Tuple Identifiers) to the rows that contain it. These references are stored in one of two formats depending on volume:

Storage Type	Condition	Format
Posting List	Few TIDs	Compact binary list stored inline within the entry
Posting Tree	Many TIDs (threshold: 256 bytes)	Separate B-tree structure



Supported Operator Classes

array_ops — for array types:

```
@> left array contains right: '{1,2,3}' @> '{1,2}'
<@ left array is contained by right
&& arrays overlap (have common elements)
= equality
```

jsonb_ops — for JSONB (default):

Indexes all keys and values.

```
jsonb @> '{"status":"active"}'
jsonb ? 'key' -- does key exist?
jsonb ?| array -- does any of these keys exist?
jsonb ?& array -- do all of these keys exist?
```

tsvector_ops — for full-text search:

```
to_tsvector('postgresql rocks') @@ to_tsquery('postgresql')
```

Each lexeme is indexed individually.

Query Execution Flow

```
tags @> {'postgres'}
Query
```

Entry Tree

"postgres"

Posting List

```
TID(1)
TID(2)
TID(4)
```

Heap Table

```
Row 1 -> {go, postgres}
Row 2 -> {python, postgres}
Row 4 -> {postgres}
```

Return Rows

Pending List (fastupdate)

A significant challenge with GIN is write overhead: inserting a single row containing {go, python, rust, java} requires **4 separate entry tree operations**.

To address this, GIN uses a **pending list** mechanism:

1. New inserts are not written directly to the main index.
2. They are first appended to a pending list (pg_gin_pending_list).
3. When VACUUM runs, or when the pending list exceeds gin_pending_list_limit (default: **4MB**), all pending entries are flushed to the main index in bulk.

This behavior is controlled by the fastupdate parameter (default: on).

```
-- Disable fastupdate for faster reads at the cost of slower writes
CREATE INDEX idx ON t USING gin(col) WITH (fastupdate = off);
```

Trade-off:

Setting	Write Speed	Read Speed
fastupdate = on (default)	Faster	Slightly slower (pending list must also be scanned)
fastupdate = off	Slower	Faster

```
INSERT {go, python, rust}
INSERT
```

Pending List
(fastupdate)

Trigger flush when:

- VACUUM runs
- pending list size > 4MB

Flush

Entry Tree update

- go Posting List
- python Posting List
- rust Posting List

Partial GIN Index

It is not always necessary to index the entire table. A WHERE clause can limit which rows are indexed:

```
CREATE INDEX idx_gin_tags ON posts USING gin(tags)
WHERE status = 'published';
```

Only rows where status = 'published' are indexed — resulting in a smaller index and faster writes.

Key Configuration Parameters

Parameter	Default	Description
gin_pending_list_limit	4MB	Maximum size of the pending list. Increasing it reduces flush frequency (faster writes) but means more pending entries to scan at query time.
fastupdate	on	Controls whether the pending list mechanism is used. Can be set at index creation or changed with ALTER INDEX.
gin_fuzzy_search_limit	0 (unlimited)	Limits the number of TIDs returned during full-text search. Useful for performance on large tables.

GIN vs B-tree

Feature	GIN	B-tree
Best suited for	Arrays, JSONB, tsvector	Simple scalar values
Supported operators	@>, &&, @@, ?	=, <, >, BETWEEN
Index size	Larger (each component indexed separately)	Smaller
INSERT speed	Slower (without pending list)	Faster
SELECT speed	Very fast (component lookup)	Fast
Range queries	No	Yes

When to Use GIN

Use GIN when:

- Searching within JSONB columns by key or value
- Using @> or && operators on array columns
- Implementing full-text search with tsvector
- Using LIKE / ILIKE with the pg_trgm extension

Avoid GIN when:

- Write-heavy, read-light workloads (pending list flush overhead accumulates)
- Simple equality or range queries (B-tree is more appropriate)
- Large JSONB documents that are rarely queried (GiST may be more suitable)

3. GiST Index (Generalized Search Tree)

Overview

GiST (Generalized Search Tree) is an extensible index infrastructure in PostgreSQL that unifies classical tree-based index structures — such as B-tree and R-tree — under a single framework.

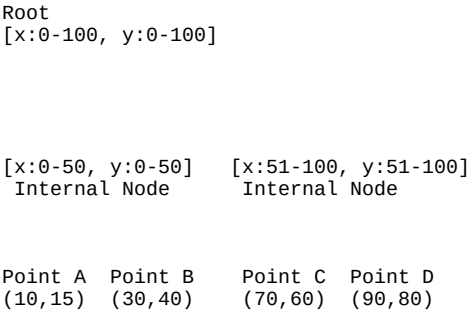
While a B-tree index handles only scalar value comparisons, GiST allows you to define your own comparison logic for any data type. GiST manages the tree structure; you provide the operator class.

Internal Structure

Internally, GiST is a **balanced tree** (similar to B-tree). However, instead of storing scalar values at each node, it stores **predicates** — conditions that describe what the subtree beneath that node contains.

Core concept: Bounding Box

Each internal node stores the minimal bounding predicate that covers all of its children. For geometry, this is the smallest rectangle enclosing all points in the subtree. During a search, if the query does not intersect a node's bounding box, that entire subtree is **pruned**.



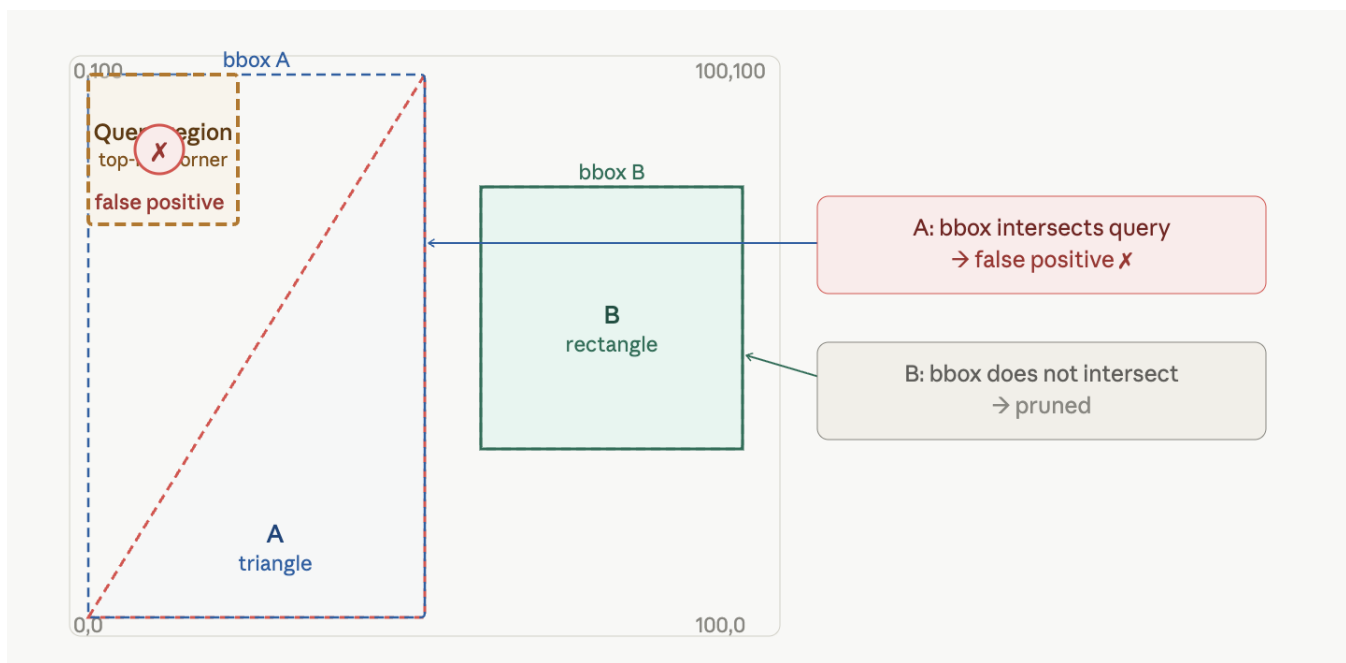
For a query at x:60, y:70 — the left subtree does not intersect the bounding box and is pruned. Only the right subtree is traversed.

Lossy vs Lossless

One of GiST's key characteristics is that it **can be lossy**.

Mode	Description
Lossless	Index result is 100% accurate — no heap recheck required.
Lossy	Index returns "possibly matching" rows. PostgreSQL fetches the actual row from the heap and rechecks the condition. Rows that do not satisfy the condition are discarded (false positives).

GiST becomes lossy when **bounding boxes overlap**. Two geometries may have overlapping bounding boxes without actually intersecting — the index returns both candidates, and the recheck discards the false match.



Supported Operator Classes

Geometry (PostGIS / built-in: point, box, polygon, circle):

```
&&    bounding boxes intersect
@>    left contains right
<@    left is contained by right
~=    same bounding box
<<    strictly to the left
>>    strictly to the right
```

Range types (int4range, tsrange, daterange, etc.):

```
&&    ranges overlap
@>    range contains element or other range
<<    strictly to the left of
>>    strictly to the right of
-|-    ranges are adjacent
```

Full-text search (tsvector):

```
@@    query matches document
```

Uses a **signature** (lossy hash) rather than exact lexemes — smaller index size compared to GIN, but lossy.

pg_trgm (trigram similarity):

```
LIKE, ILIKE, ~ (regex)
```

Produces a smaller index than GIN with pg_trgm, at the cost of slightly slower lookups.

cube extension — for multi-dimensional cubes, including KNN search.

hstore — key-value storage: @>, ?, ?|, ?&

inet — IP addresses: && (network overlap), >>, << (subnet containment)

Query Execution Flow

Query Area
[x:60-80, y:60-80]

Root
[x:0-100, y:0-100]

[x:0-50, y:0-50] No overlap Pruned	[x:51-100, y:0-100] Overlap Traversed
--	---

Candidate Rows

Heap Recheck

Return Rows

KNN (K-Nearest Neighbor) Search

One of GiST's strongest advantages over GIN is support for **distance-based ordering**:

```
-- Find the 5 nearest locations
SELECT * FROM locations
ORDER BY position <-> point(40.4, 49.8)
LIMIT 5;
```

The <-> operator computes distance. GiST executes this as an **ordered index scan** — it finds the nearest points directly through the index without scanning the full table.

Exclusion Constraints

A unique capability of GiST is support for **EXCLUDE constraints** — something B-tree cannot provide for non-equality operators.

```
-- Prevent overlapping reservations for the same room
CREATE TABLE reservations (
    room_id int,
    during tsrange,
    EXCLUDE USING gist (room_id WITH =, during WITH &&)
);
```

This constraint ensures that no two rows can have the same room_id with overlapping during ranges. GiST enforces this efficiently on every INSERT. This is not possible with a B-tree index.

VACUUM and GiST

Unlike GIN, GiST has **no pending list**. Every INSERT is written directly to the index, which means:

- More work per write (splits may occur)
- No pending list to scan at query time

VACUUM removes dead tuples and reclaims empty pages. However, GiST **does not rebalance** the tree during VACUUM — it only cleans up dead pointers.

On tables with frequent DELETE operations, GiST indexes can become bloated. A periodic REINDEX may be necessary.

GiST vs GIN

Feature	GiST	GIN
Accuracy	Can be lossy (heap recheck required)	Always lossless
Index size	Smaller	Larger
Write speed	Faster	Slower (without pending list)
Read speed	Slightly slower	Faster
KNN / distance search	Yes	No
Geometry	Yes — primary choice	Not supported
Full-text search	Possible (smaller index)	Yes — primary choice
Range types	Yes	Limited
LIKE / trigram	Possible (smaller index)	Possible (faster lookups)

Practical guidelines:

- Geometry, range types, inet, or KNN queries **GiST**
- Full-text search, JSONB, array @> **GIN**
- LIKE queries with smaller index size **GiST + pg_trgm**
- LIKE queries with faster lookups **GIN + pg_trgm**

4. BRIN Index (Block Range Index)

Overview

BRIN (Block Range Index) is the smallest, simplest, and most specialized index type in PostgreSQL.

The core idea: if there is a strong correlation between a column's values and the physical order of rows on disk, there is no need to index every individual value. Instead, BRIN stores the **minimum and maximum values** for each group of consecutive disk blocks (a block range).

Example: Consider a created_at column. New rows are always appended at the end — meaning their timestamps are physically ordered. BRIN records: "blocks 1–128 contain created_at values between 2024-01-01 and 2024-03-15." A query searching for 2024-06-01 can skip that entire block range without even reading it.

Internal Structure

BRIN divides the entire table into groups of pages_per_range pages (default: **128 pages**). For each group, it stores a **summary**.

By default, the summary contains:

Field	Description
min	The smallest value in the range
max	The largest value in the range
hasnulls	Are there any NULL values?
allnulls	Are all values NULL?

Range 1 (Pages 1 - 128)
min(created_at) = 2024-01-01
max(created_at) = 2024-03-31

Range 2 (Pages 129 - 256)
min(created_at) = 2024-04-01
max(created_at) = 2024-06-30

Range 3 (Pages 257 - 384)
min(created_at) = 2024-07-01
max(created_at) = 2024-09-30

How a query uses BRIN:

BRIN scans the summary for each range. If the query value falls outside a range's min–max interval, that entire range (128 pages) is skipped. If it falls within the interval, those pages are read and rechecked (lossy — similar to GIST).

Range 1 (pages 1-128):	min=1, max=1000	query=500	READ
Range 2 (pages 129-256):	min=1001, max=2000	query=500	SKIP
Range 3 (pages 257-384):	min=2001, max=3000	query=500	SKIP

Index Size — The Primary Advantage

BRIN's most significant advantage is its remarkably small size.

For a table with 100 million rows:

Index Type	Approximate Size
B-tree	~2–3 GB
BRIN	~48 KB

Why? Because BRIN does not index individual rows. It represents each group of ~128 pages (~1 MB of data) with just a few bytes. The entire index typically fits in memory, minimizing disk I/O.

Correlation — The Key Requirement

BRIN is only effective when there is **high correlation** between a column's values and the physical row order on disk.

High correlation (BRIN works well):

- created_at, inserted_at — increases with insert order
- id (serial / sequence) — always increasing
- IoT sensor data — arrives in time order
- Log tables — new entries are always appended

Low correlation (BRIN is ineffective):

- email, username — inserted in random order
- status, category — few distinct values distributed everywhere
- Columns with frequent UPDATE — value changes but physical location does not

Checking correlation:

```
SELECT correlation
FROM pg_stats
WHERE tablename = 'orders' AND attname = 'created_at';
-- Close to 1.0  BRIN is highly effective
-- Close to 0.0  BRIN is not suitable
```

pages_per_range Parameter

The most important BRIN configuration parameter:

```
CREATE INDEX ON orders USING brin(created_at) WITH (pages_per_range = 64);
```

Value	Range Count	Index Size	Precision	Scan Speed
Small (e.g. 16)	More ranges	Larger	Higher — fewer false positives	Slower
Large (e.g. 256)	Fewer ranges	Smaller	Lower — more false positives, more rechecks	Faster

The default value of **128** is a good starting point in most cases. If column correlation is very high, increasing to 256 is reasonable. For moderate correlation, values between 32 and 64 typically yield better results.

Bloom Filter BRIN (PostgreSQL 14+)

Starting with PostgreSQL 14, BRIN supports **bloom filter operator classes** for columns with low correlation:

```
-- Standard BRIN — NOT suitable for low-cardinality / low-correlation columns
CREATE INDEX ON orders USING brin(status);

-- Bloom filter BRIN — works even without correlation
CREATE INDEX ON orders USING brin(status brin_bloom_ops(false_positive_rate = 0.05));
```

Instead of storing min/max values, the bloom filter stores a **bit array** per range. It probabilistically answers: *"does this range contain this value?"*

Result	Meaning
"Yes" (possibly)	Value may exist — heap recheck required (false positives possible)
"No"	Value definitely does not exist — range is skipped (no false negatives)

This makes BRIN useful for low-correlation columns, though it is less precise than GIN or B-tree.

Autosummarization

As new data is inserted, new pages are created. Whether BRIN automatically summarizes these new ranges depends on the autosummarize parameter:

```
CREATE INDEX ON logs USING brin(created_at) WITH (autosummarize = on);
```

Setting	Behavior
autosummarize = on (default)	autovacuum automatically summarizes new unsummarized ranges. Queries always see the full index.
autosummarize = off	New ranges are not summarized — those pages are scanned sequentially at query time. Manual summarization is required. Useful after bulk inserts.

Manual summarization functions:

```
-- Summarize only new (unsummarized) ranges
SELECT brin_summarize_new_values('orders_created_at_idx');

-- Recompute all ranges (after UPDATE / DELETE)
SELECT brin_summarize_range('orders_created_at_idx', 0);

-- Remove summary for a specific range
SELECT brin_desummarize_range('orders_created_at_idx', 0);
```

The UPDATE Problem

BRIN has a subtle weakness: **UPDATES corrupt the summary**.

When a row is updated, PostgreSQL marks the old row as dead and writes the new row — potentially to a different block (if a HOT update is not possible). As a result, the block range summary may no longer be accurate: the recorded max may be higher than the actual maximum, or the min may be lower than the actual minimum.

This increases **false positives** — BRIN reports more ranges as "possibly matching," leading to more rechecks and degraded query performance.

VACUUM partially resolves this: after dead tuples are removed, brin_summarize_range can recompute correct min/max values. However, on tables with frequent UPDATE operations, BRIN's effectiveness degrades over time.

BRIN vs Other Index Types

Feature	BRIN	B-tree	GIN	GiST
Index size	Very small (KB)	Medium (GB)	Large	Medium
Build speed	Very fast	Medium	Slow	Medium
SELECT accuracy	Lossy (recheck)	Lossless	Lossless	Can be lossy
Correlation required	Yes — high	No	No	No
Range queries	Yes	Yes	No	Yes
Equality queries	Weak	Excellent	Excellent	Moderate
KNN search	No	No	No	Yes
UPDATE impact	Summary corrupted	Normal	Normal	Normal
Best for	Large append-only tables	General purpose	Arrays / JSONB / FTS	Geometry / Ranges

When to Use BRIN

Use BRIN when:

- Very large tables (hundreds of millions of rows) where disk space is a concern
- Append-only or rarely updated tables
- Columns with high physical correlation: created_at, id, time-series data
- Data warehouses, log tables, IoT data, audit tables
- pg_stats.correlation > 0.9 for the target column

Avoid BRIN when:

- The table is small (B-tree will be faster)
- The column has random insert order
- The column is frequently updated
- High precision is required (recheck overhead becomes a bottleneck)
- Point lookups (WHERE id = 12345) — B-tree is significantly faster

5. Partial Index

Overview

A **Partial Index** is an index defined with a WHERE predicate. Only rows that satisfy the predicate are included in the index structure — all other rows are completely ignored.

```
CREATE INDEX idx_orders_pending ON orders(created_at)
WHERE status = 'pending';
```

This means the index only tracks pending orders. Rows where status = 'completed' are never written to the index, never maintained, and consume no index space.

Physical Structure

A partial index is a **separate index file** — physically identical in format to a full index. The only difference is that it contains far fewer entries.

Relationship with the heap:

Like any index, a partial index stores a **TID (Tuple Identifier)** — a (page_number, offset) pair — for each entry. This TID is a physical pointer to the corresponding row in the heap.

Partial Index (B-tree):

```
created_at=2025-01-01  TID(3,2)      status='pending' row
created_at=2025-02-15  TID(7,1)      status='pending' row
created_at=2025-04-10  TID(9,4)      status='pending' row
```

```
Heap (table):
Page 1: (id=1, status='completed', ...) NOT in index
Page 3: (id=2, status='pending', ...) IN index TID(3,2)
Page 5: (id=3, status='completed', ...) NOT in index
Page 7: (id=4, status='pending', ...) IN index TID(7,1)
```

completed rows exist physically in the heap — the partial index simply holds no pointer to them.

Size Advantage

The size difference between a full index and a partial index can be dramatic:

Full index (all rows):	Partial index (WHERE status='pending'):
B-tree root page 1: (pending + completed rows) page 2: page 848: ... 10M rows ~2 GB	B-tree root page 1: (pending rows only) page 2: 50K rows ~3 MB

Same B-tree structure — significantly fewer nodes and pages.

DML Behavior

INSERT

When a new row is inserted, PostgreSQL **evaluates the predicate**:

```
INSERT INTO orders(status, created_at) VALUES ('pending', NOW());
-- predicate: status = 'pending' TRUE added to index

INSERT INTO orders(status, created_at) VALUES ('completed', NOW());
-- predicate: status = 'pending' FALSE NOT added to index
```

Predicate evaluation is very cheap — a simple comparison. For rows that do not satisfy the predicate, there is **zero index maintenance**: no B-tree split, no page write, no WAL entry for the index. This is the primary reason partial indexes improve write performance.

UPDATE

UPDATE behavior depends on how the predicate is affected:

Scenario	Description
Predicate unchanged, value changes	Old TID removed, new TID added to index
Predicate TRUE FALSE (e.g. pending completed)	Entry removed from index; dead entry cleaned by VACUUM
Predicate FALSE TRUE (e.g. completed pending)	New TID added to index
Predicate unchanged, indexed column unchanged	HOT (Heap Only Tuple) update may occur — index is not touched at all

HOT updates occur more frequently with partial indexes because fewer columns are indexed, reducing the chance of touching the index on every update.

DELETE

```
DELETE FROM orders WHERE id = 4;
-- status='pending' TID(7,1) exists in index
-- MVCC: heap tuple is marked dead
-- Index entry remains temporarily – points to dead tuple
-- VACUUM removes both the dead heap tuple and the index entry
```

When completed rows are deleted, the index is not touched — they were never in it.

Query Planning — Predicate Implication

The planner uses a partial index only when it can **prove** that the query's WHERE clause implies the index predicate. This is called **predicate implication checking** (handled in PostgreSQL's `pretest.c` module).

```
-- Index predicate: status = 'pending'

-- Query 1:
WHERE status = 'pending' AND created_at > '2025-01-01'
-- Predicate is satisfied index is used

-- Query 2:
WHERE created_at > '2025-01-01'
-- No status condition predicate cannot be proven index is NOT used

-- Query 3:
WHERE status IN ('pending', 'failed')
-- Covers more than just 'pending' implication fails index is NOT used
```

How implication checking works:

- 1. Decompose the query's WHERE clause into individual conditions
- 2. Compare each condition against the index predicate
- 3. If the predicate is a logical consequence of the query conditions index can be used

```
Query condition: status = 'pending' AND created_at > '2025-01-01'
Index predicate: status = 'pending'

Predicate is a subset of the query condition implication holds index is used
```

Scan Types

A partial index can be accessed via three scan methods:

Scan Type	When Used
Index Scan	Small number of matching rows — fetches each TID from the index, then reads from heap
Bitmap Index Scan	Moderate number of rows — collects all TIDs first, then reads heap pages in order (sequential, not random)
Index Only Scan	All required columns are in the index — heap access is skipped. However, the visibility map is still checked; if a page is not "all-visible", a heap visit is required to verify row visibility (not to read values)

Valid Predicate Expressions

Partial index predicates must be **immutable** — they must not change after the index is created.

Supported:

```
WHERE status = 'pending'           -- equality
WHERE deleted_at IS NULL           -- IS NULL check
WHERE is_active = true              -- boolean
WHERE amount > 1000                 -- range
WHERE status IN ('a', 'b')         -- IN list
WHERE created_at > '2024-01-01'    -- fixed literal
```

Not supported:

```
WHERE created_at > NOW()            -- dynamic function
WHERE created_at > NOW() - INTERVAL '7d' -- dynamic expression
```

`NOW()` returns a different value on every call — it is not immutable and cannot be used as a predicate.

VACUUM and Partial Indexes

`VACUUM` processes a partial index the same way it processes a full index — but with significantly less work:

1. Identifies dead tuples in the heap
2. Finds index entries pointing to those dead tuples
3. Because completed rows are not in the index, `VACUUM` does not touch index entries for them
4. Only cleans up index entries for dead pending rows

This reduces `VACUUM` overhead, especially when the predicate column changes infrequently.

WAL Behavior

Every change written to a partial index is also written to the **WAL (Write-Ahead Log)** — the same as a full index. The difference is that fewer rows enter the index, so fewer WAL records are generated. This reduces replication traffic and recovery time.

When to Use Partial Indexes

Use a partial index when:

- Only a small subset of rows is queried frequently (e.g. `status = 'pending'`, `deleted_at IS NULL`)
- The predicate column has low cardinality but queries target a specific value
- You want to enforce a **conditional unique constraint** (e.g. unique active users)
- You want to reduce index size and write overhead on large tables

Avoid partial indexes when:

- Queries do not include the predicate condition — the planner cannot use the index
 - The predicate matches most rows — a full index would be equally efficient
 - The predicate uses dynamic expressions (`NOW()`, parameters)
-

6. Composite Index

Overview

A **Composite Index** (also called a multi-column index) is an index that covers **two or more columns** within a single index structure.

```
CREATE INDEX idx_orders ON orders(user_id, status, created_at);
```

Instead of creating separate indexes for each column, a composite index stores all key columns together — allowing the planner to satisfy multiple filter conditions with a single index lookup.

Internal Structure

In a composite index built on B-tree, each index entry stores **all key columns** concatenated together, followed by a **TID (Tuple Identifier)** — a (page_number, offset) pointer to the corresponding heap row.

Each index entry:
(user_id, status, created_at)

Root

1		active		2025-01-01	TID(3,2)
1		active		2025-03-15	TID(7,1)
1		pending		2025-02-01	TID(4,3)
2		active		2024-12-01	TID(9,1)
2		cancelled		2025-01-10	TID(2,5)

Sort order: Entries are sorted first by user_id, then by status within each user_id, then by created_at within each (user_id, status) pair. This is **lexicographic ordering** — analogous to how words are sorted in a dictionary.

The Leftmost Prefix Rule

This is the most important principle of composite indexes — and it follows directly from the physical structure of B-tree.

For an index on (a, b, c), the index can only be used for queries that reference columns **starting from the leftmost column in order**.

Why? Because b values are not globally sorted — they are sorted within each value of a. Without knowing a, the B-tree has no starting point for the search.

Index: (user_id, status, created_at)

USED

user_id

used

user_id + status

used

user_id + status + created_at

used

NOT USED

status

not used

created_at

not used

status + created_at

not used

DML Behavior

INSERT

When a new row is inserted, PostgreSQL forms the composite key from all indexed columns and places it in the correct position in the B-tree:

```
INSERT INTO orders(user_id, status, created_at)
VALUES (1, 'pending', NOW());
```

1. Key (user_id=1, status='pending', created_at=2025-06-03) is formed
2. The correct leaf node is located in the B-tree
3. The entry is inserted into the leaf node
4. If the node is full, a **split** occurs — the node is divided into two, and the parent node is updated
5. The new TID is written to the index entry

UPDATE

PostgreSQL never modifies an index entry in place. Every UPDATE is effectively a **delete + insert**:

```
UPDATE orders SET status = 'completed' WHERE id = 42;
```

1. The old heap tuple is marked as dead (MVCC)
2. The old index entry remains temporarily — it now points to a dead tuple
3. A new heap tuple is written
4. A new index entry (user_id=1, status='completed', created_at=...) with the new TID is added
5. VACUUM later removes the dead heap tuple and the stale index entry

HOT (Heap Only Tuple) Update is an exception: if the modified column is not part of any index, PostgreSQL updates only the heap without touching the index. As a composite index covers more columns, the probability of a HOT update decreases — further increasing write overhead.

```
UPDATE orders SET notes = 'urgent' WHERE id = 42;
-- 'notes' is not in any index  HOT update  index is not touched
```

```
UPDATE orders SET status = 'completed' WHERE id = 42;
-- 'status' is in the composite index  HOT not possible  index is updated
```

DELETE

```
DELETE FROM orders WHERE id = 42;
```

1. The heap tuple is marked as dead (MVCC)
2. The index entry remains temporarily — pointing to the dead tuple
3. VACUUM removes both the dead heap tuple and the corresponding index entry

Scan Types

Index Scan — used when a small number of rows are expected:

```
(user_id=1, status='active')  TID(3,2)  read from heap
                             TID(7,1)  read from heap
                             TID(9,4)  read from heap
```

Each TID results in a random heap access. Efficient for few rows; causes excessive random I/O for many rows.

Bitmap Index Scan — used for a moderate number of rows:

1. Collect all matching TIDs build a bitmap
TID(3,2), TID(7,1), TID(9,4), TID(2,8) ...
2. Sort the bitmap by page number
3. Read heap pages sequentially (no random I/O)

Eliminates random I/O by batching and ordering heap reads.

Index Only Scan — used when all SELECT columns are present in the index:

```
-- Index: (user_id, status, created_at)
SELECT user_id, status, created_at FROM orders WHERE user_id = 1;
-- All required columns are in the index heap is not accessed
```

Column Order and Range Conditions

This is one of the most commonly misunderstood aspects of composite indexes.

Problem: A range condition (>, <, BETWEEN) on a column prevents the index from filtering subsequent columns.

Why? A range condition selects an interval of entries. Within that interval, the next column's values are not globally sorted — they are sorted within each value of the preceding column.

```
-- Index: (user_id, created_at, status)
WHERE user_id = 1 AND created_at > '2025-01-01' AND status = 'active'
```

```
B-tree traversal:
user_id=1 found
  created_at > '2025-01-01' range scan begins
    (user_id=1, created_at=2025-01-02, status='pending') status rechecked
    (user_id=1, created_at=2025-01-02, status='active') status rechecked
    (user_id=1, created_at=2025-02-10, status='completed') status rechecked
    ...
```

status is not filtered by the index — every entry within the range is rechecked.

Solution — place equality columns first, range column last:

```
-- Index: (user_id, status, created_at)
WHERE user_id = 1 AND status = 'active' AND created_at > '2025-01-01'
```

```
B-tree traversal:
user_id=1 found
  status='active' found (exact match)
    created_at > '2025-01-01' range scan within (user_id=1, status='active') only
```

Far fewer entries are examined.

INCLUDE — Non-Key Columns (PostgreSQL 11+)

There are two ways to add a column to an index:

```
-- Option 1: as a key column
CREATE INDEX ON orders(user_id, status, created_at, total_amount);

-- Option 2: with INCLUDE
CREATE INDEX ON orders(user_id, status) INCLUDE (created_at, total_amount);
```

What is the difference?

Key columns are stored in **all B-tree nodes** — both internal nodes (for routing) and leaf nodes. They participate in searching, sorting, and filtering.

INCLUDE columns are stored **only in leaf nodes** — solely to enable Index Only Scans. They are not written to internal nodes, do not increase internal node size, and do not trigger splits.

B-tree with key columns only:
Internal node:
(user_id=1, status='active')
(user_id=2, status='pending')
total_amount also here

Leaf node:
(1, 'active', 2025-01-01, 150.00, TID)

B-tree with INCLUDE:
Internal node:
(user_id=1, status='active')
(user_id=2, status='pending')
total_amount absent, node is smaller

Leaf node:
(1, 'active') INCLUDE(2025-01-01, 150.00) TID

INCLUDE columns **cannot be used in WHERE conditions** — they are only available in SELECT.

```
CREATE INDEX ON orders(user_id, status) INCLUDE (total_amount);

SELECT total_amount FROM orders WHERE user_id=1 AND status='active';
-- Index Only Scan — heap not accessed

SELECT * FROM orders WHERE total_amount > 100;
-- Index not used for filtering
```

ORDER BY and Composite Indexes

A composite index can also serve sorting — if the ORDER BY clause matches the index column order. When it does, no separate **Sort node** is added to the query plan.

```
CREATE INDEX ON orders(user_id ASC, created_at DESC);

ORDER BY user_id ASC, created_at DESC -- matches index direction exactly
ORDER BY user_id DESC, created_at ASC -- full reverse — B-tree supports this too

ORDER BY user_id ASC, created_at ASC -- mixed direction — Sort node required
ORDER BY user_id DESC, created_at DESC -- mixed direction — Sort node required
```

Composite Index vs Separate Indexes

```
-- Option A: separate indexes
CREATE INDEX ON orders(user_id);
CREATE INDEX ON orders(status);

-- Option B: composite index
CREATE INDEX ON orders(user_id, status);
```

With Option A, PostgreSQL can use **Bitmap Index Scan + BitmapAnd**:

```
BitmapAnd
  Bitmap Index Scan on idx_user_id (collect TIDs where user_id=1)
  Bitmap Index Scan on idx_status (collect TIDs where status='active')
  Intersect the two bitmaps
```

This requires two separate scans plus an AND operation. Option B directly locates (user_id=1, status='active') in a single scan — faster and more efficient.

Scenario	Recommendation
Queries always filter both columns together	Composite index
Columns are queried independently in different queries	Separate indexes

Bloat

In tables with frequent UPDATE operations, composite indexes are susceptible to **index bloat**:

- Each UPDATE marks the old entry as dead and inserts a new one
- Over time, dead entries accumulate

3. The physical index size grows beyond what live data requires
 4. Queries read unnecessary pages
-

When to Use a Composite Index

Use a composite index when:

- Queries consistently filter on multiple columns together
- Column order can be arranged to satisfy the leftmost prefix rule
- You want to support Index Only Scans by including all selected columns
- You want to eliminate a Sort node by matching ORDER BY column order

Avoid composite indexes when:

- Columns are queried independently in different queries (separate indexes may be better)
- The index covers too many columns, increasing write overhead significantly
- The leftmost column has very low cardinality and is rarely used as a filter